

Day 2: Mappings, Transformations & Mapping Tasks

Welcome to Day 2. Today we move from setup to action — learning how to move, shape, and deliver data exactly where it needs to go. Think of yourself as a data traffic controller: data arrives raw, you route it, clean it, transform it, and send it to its destination in perfect form.

DATA INTEGRATION · ETL ENGINEERING · HANDS-ON



Today's Agenda

What We're Covering Today

Eleven focused sessions, building from foundational concepts to real execution. Each topic builds on the last — by the end of today, you'll have designed, run, and monitored your own data pipeline.

01

Mapping Concepts

The blueprint of data movement

02

Mapping Designer

Building mappings hands-on

03

Data Types & Field Handling

Compatibility and conversions

04

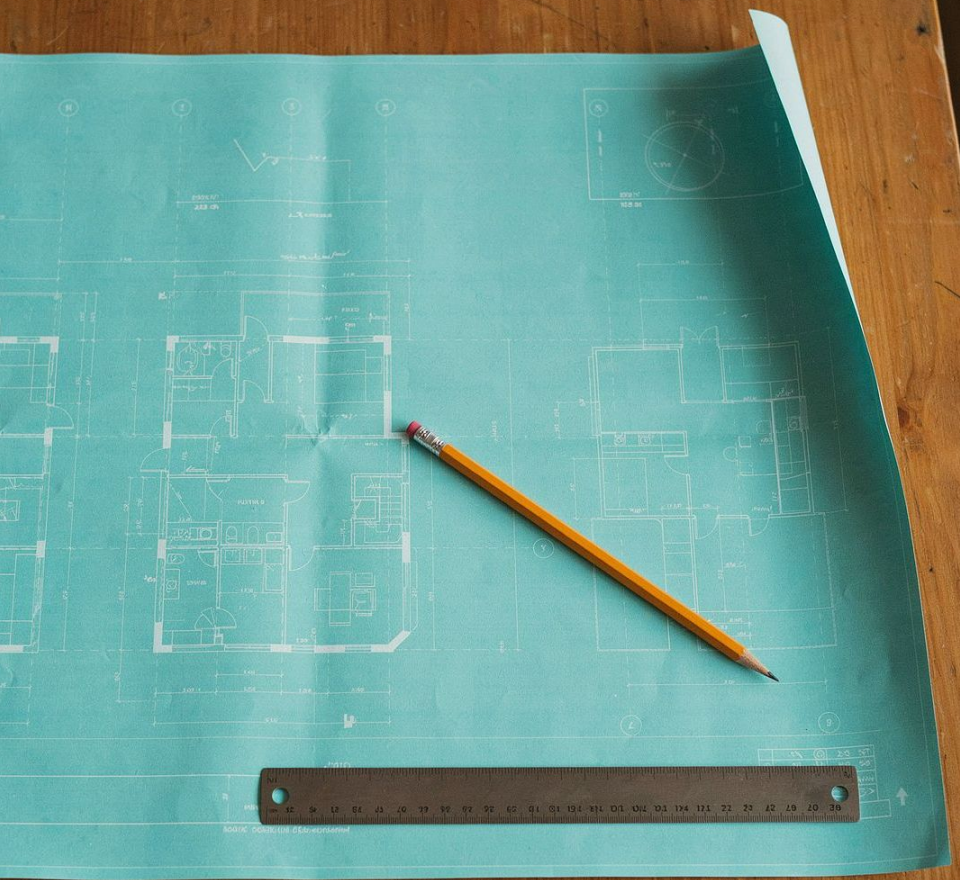
Transformations

Expression, Filter, Lookup, Aggregator

05

Mapping Tasks & Monitoring

Execute, observe, troubleshoot

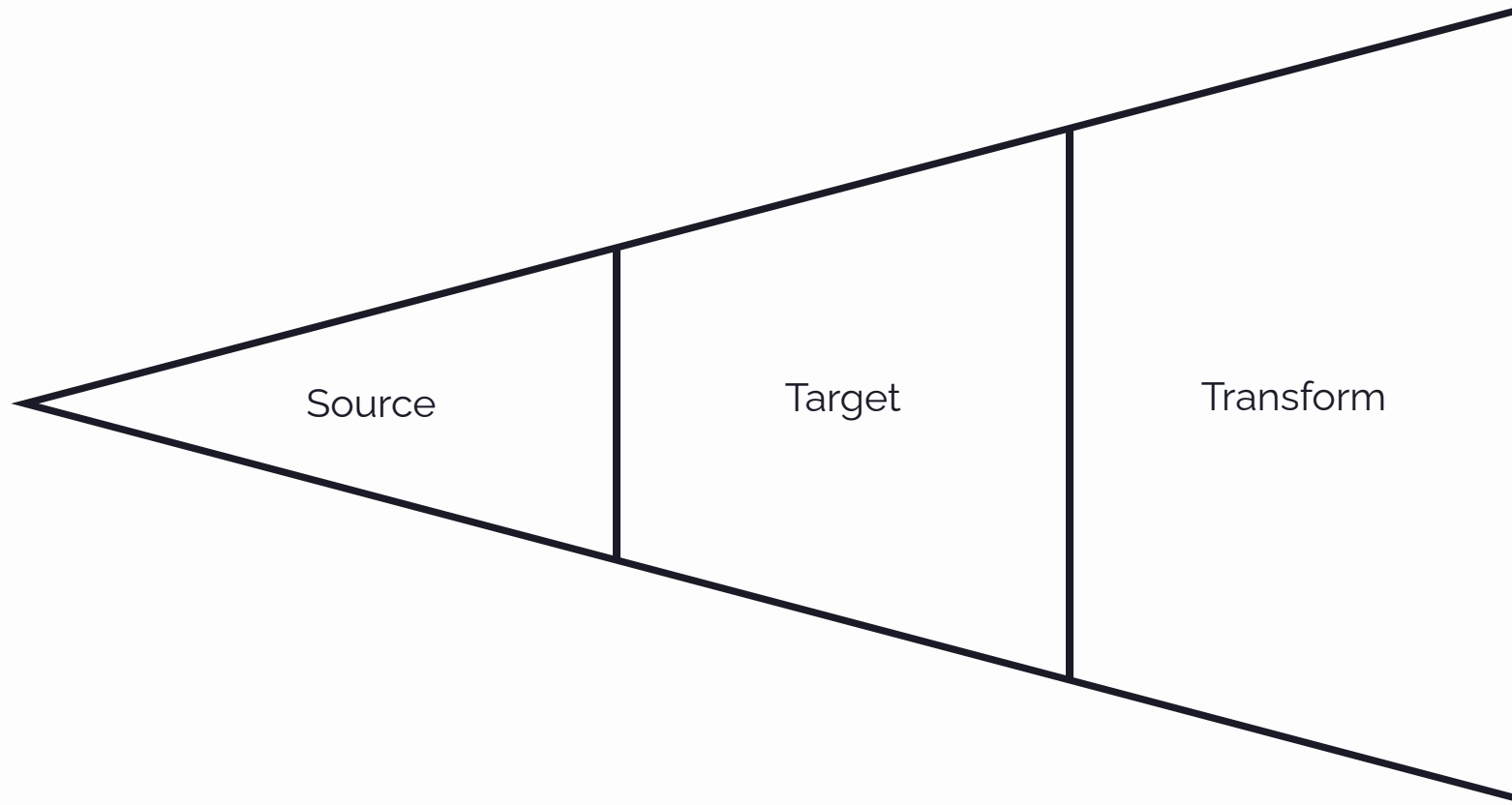


Session 1: Mapping Concepts

Before touching a tool, let's agree on what a **mapping** actually is. Think of it as the architectural blueprint for your data pipeline — it defines *what* data comes in, *how* it gets transformed, and *where* it lands. A mapping does not run on its own; that's the job of a **Mapping Task**. Understanding this separation is fundamental to thinking like a reusable, scalable ETL developer.

The Source → Transform → Target Flow

Every mapping follows the same fundamental pattern, no matter how complex the logic inside. Picture an online retailer processing daily orders: raw order records come in from a database, get cleaned and enriched in the middle, and land in a reporting warehouse at the end.



This three-stage mental model applies to every mapping you will ever build. Keep it front of mind as complexity grows.

Mapping vs. Mapping Task

One of the most common points of confusion for new developers. Here's a clean analogy: a **mapping** is a recipe — it describes the ingredients and steps. A **Mapping Task** is actually cooking the meal — it executes the recipe in a specific runtime environment with specific source and target connections.

Mapping

- The design-time blueprint
- Defines sources, targets, transformations
- Reusable across multiple tasks
- No runtime configuration here

Mapping Task

- The runtime execution unit
- References a specific mapping
- Assigns connections and runtime environment
- What you schedule and monitor

📌 Design once, run many times. A single mapping can power multiple tasks — the reusable development mindset saves enormous time at scale.

Session 2: Mapping Designer

The Mapping Designer is your canvas. You drag, connect, and configure transformations visually — no code required for most tasks. Let's walk through the core steps of building your first mapping, from choosing your source connection all the way to confirming your target fields are correctly mapped.



Building a Mapping: Step by Step

Here's how a typical mapping comes together in the designer. Imagine loading a CSV file of new customer records into a Salesforce CRM target — a real-world scenario we'll use throughout today's sessions.

1

Select Source Connection

Choose the connection to your source system — a flat file, database, or cloud app.

2

Configure Source Transformation

Select the specific object or file. Metadata (field names, data types) is auto-imported.

3

Add Transformations

Insert logic between source and target — filter rows, derive columns, join data.

4

Configure Target Transformation

Choose the destination object and define the write operation (insert, update, upsert).

5

Map Fields

Connect source fields to target fields — use auto-mapping or drag manually.

Auto-Mapping vs. Manual Mapping

Field mapping is where the rubber meets the road. IICS gives you two approaches — and knowing when to use each saves significant time.

Auto-Mapping

The designer matches source and target fields automatically by **name and data type**. Perfect when your source and target schemas are aligned — for example, loading a staging table into a warehouse with identical column names. Fast, but always review the result.

Manual Mapping

You drag each source field to its target field explicitly. Required when field names differ (e.g., `cust_id` → `CustomerID`), when types need conversion, or when derived logic determines the target value. More work up front, more control always.

A collection of ten small, square wooden blocks arranged in a loose circle on a light-colored surface. Each block is a different color and has a data type written on it in white capital letters. The blocks are: a red block labeled 'STRING', a teal block labeled 'INTEGER', a yellow block labeled 'FLOAT', a green block labeled 'BOOLEAN', a purple block labeled 'ARRAY', an orange block labeled 'OBJECT', a light blue block labeled 'NULL', a pink block labeled 'CHAR', and a lime green block labeled 'LIST'.

Session 3: Data Types & Field Handling

Data types are the language that source and target systems use to describe their fields. When those languages don't match — say, a source sends a number as text — transformations fail, data gets rejected, or worse, silently corrupted. Understanding type compatibility and how to handle edge cases is a core ETL skill.

Common Data Type Challenges

In the real world, source data is rarely clean. A date field arrives as a string. A numeric price column contains a `NULL`. A phone number has formatting characters. Here's how to think about each scenario:



Date/String Mismatches

Use expression functions like `TO_DATE()` to convert string values such as `"2024-01-15"` into proper date fields your target system understands.



Handling NULLs

Use `IIF(ISNULL(field), 'Unknown', field)` to replace nulls with safe defaults, preventing downstream errors in reports or applications.



Precision & Length

A source `decimal(10,4)` field mapped to a target `decimal(8,2)` will truncate. Always check precision and scale compatibility before mapping numeric fields.

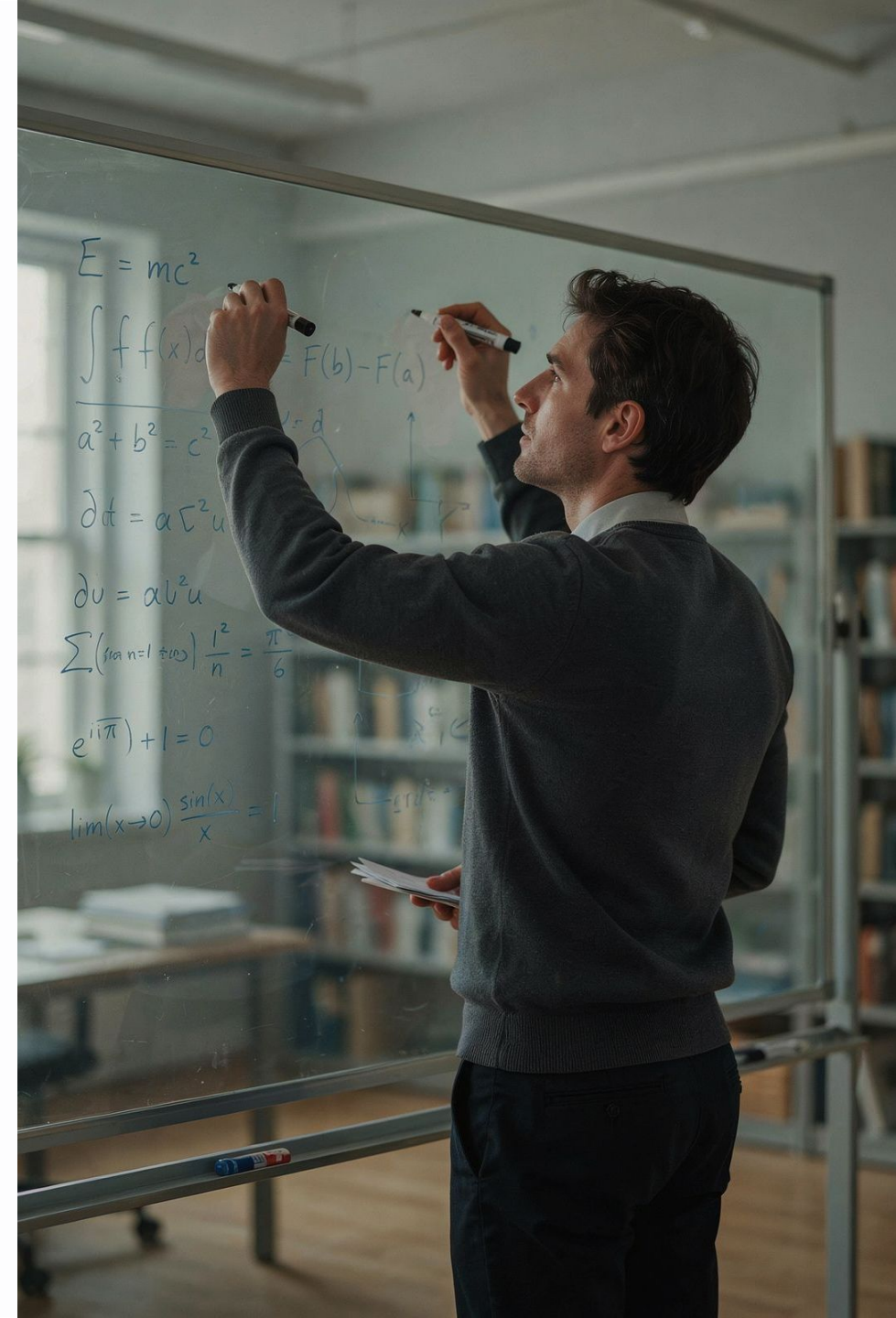


Basic Data Quality

Trim whitespace with `LTRIM(RTRIM())`, standardize case with `UPPER()` or `LOWER()`, and validate patterns early in the pipeline — not at the target.

Session 4: Expression Transformation

The Expression Transformation is your Swiss Army knife. It lets you create **derived columns** — new fields calculated from existing data — without leaving the mapping canvas. No stored procedures, no custom code in the source system. Just clean, inline logic you can read, debug, and reuse.



Expression Transformation in Practice

Let's make this concrete. You're processing an e-commerce orders table. Here are four real expression scenarios you'd encounter on the job:

String Operations

Scenario: Combine first and last name into a display name.

`CONCAT(first_name, ' ', last_name)`

Also useful: `UPPER()`, `SUBSTR()`, `LENGTH()`

Numeric Calculations

Scenario: Calculate order total with tax.

`order_subtotal * 1.08`

Use `ROUND()` to control decimal precision on currency fields.

Date Expressions

Scenario: Calculate days since last purchase.

`DATE_DIFF(SYSDATE(), last_order_date, 'DD')`

Essential for aging reports and SLA calculations.

Conditional Logic

Scenario: Flag high-value customers.

`IIF(lifetime_value > 5000, 'VIP', 'Standard')`

Nest `IIF()` calls for multiple conditions — like a series of if/else statements.



Session 5: Filter Transformation

Not every row in your source belongs in your target. The Filter Transformation acts like a **gatekeeper** — it evaluates a condition on every row and only passes through the rows that meet your business rules. Rows that fail the condition are simply dropped from the pipeline.

Real-World Example

Loading only **active customers** to a marketing platform:

`status = 'ACTIVE' AND opt_out = 'N'`

Only rows meeting both conditions pass through — inactive and opted-out customers never reach the target.

Design Tips

- Place filters **early** in the pipeline to reduce data volume before expensive lookups or aggregations
- Keep filter conditions readable — complex logic is harder to debug
- Document what gets *excluded*, not just what passes

Session 6: Lookup Transformation

The Lookup Transformation lets you **enrich your data in-flight** by looking up values from a reference dataset. Think of it like a `VLOOKUP` in Excel — but happening inside your pipeline at scale. It's one of the most commonly used transformations in real-world ETL work.



Classic use case: Your orders table contains a `product_id`. You look up the `product_name` and `category` from a products reference table, adding those fields to each order row before loading into the warehouse — no joins in the target system needed.

Session 7: Aggregator Transformation

The Aggregator Transformation answers questions like: *"What is the total sales per region?"* or *"How many orders did each customer place this month?"* It groups rows and applies aggregate functions — just like a `GROUP BY` in SQL, but configured visually inside your mapping.

Group By Fields

Define *how* to group your data. Example: group by `region` and `product_category` to get per-region, per-category summaries.

SUM

Total a numeric field within each group.

`SUM(order_total)`

Classic use: total revenue per sales rep per quarter.

COUNT

Count rows within each group.

`COUNT(order_id)`

Classic use: number of transactions per customer per month.

AVG

Average a numeric field across the group.

`AVG(deal_size)`

Classic use: average order value by product category.

- ❏ The Aggregator must come **after** filtering and lookups in your mapping flow — aggregate on the data you actually want to summarize, not the raw unfiltered input.

Session 8: Mapping Validation & Data Preview

Before you run a mapping against live data, you validate it. IICS checks your mapping for configuration errors — unconnected ports, incompatible data types, missing required fields — and flags them before execution. Think of it as a compiler check for your pipeline. Fix errors at design time, not at 2am when the job fails in production.



Validation Errors

Red indicators signal blocking issues — for example, a target field marked as required has no mapped source. The mapping will not save until these are resolved.



Data Preview

Run a live preview against a small sample of your source data. See exactly what each transformation produces row-by-row — the fastest way to verify your expression logic is correct before full execution.



Troubleshooting Tips

Work left to right: confirm your source fields look correct first, then check each transformation output in sequence. Isolate the transformation where the data first looks wrong.

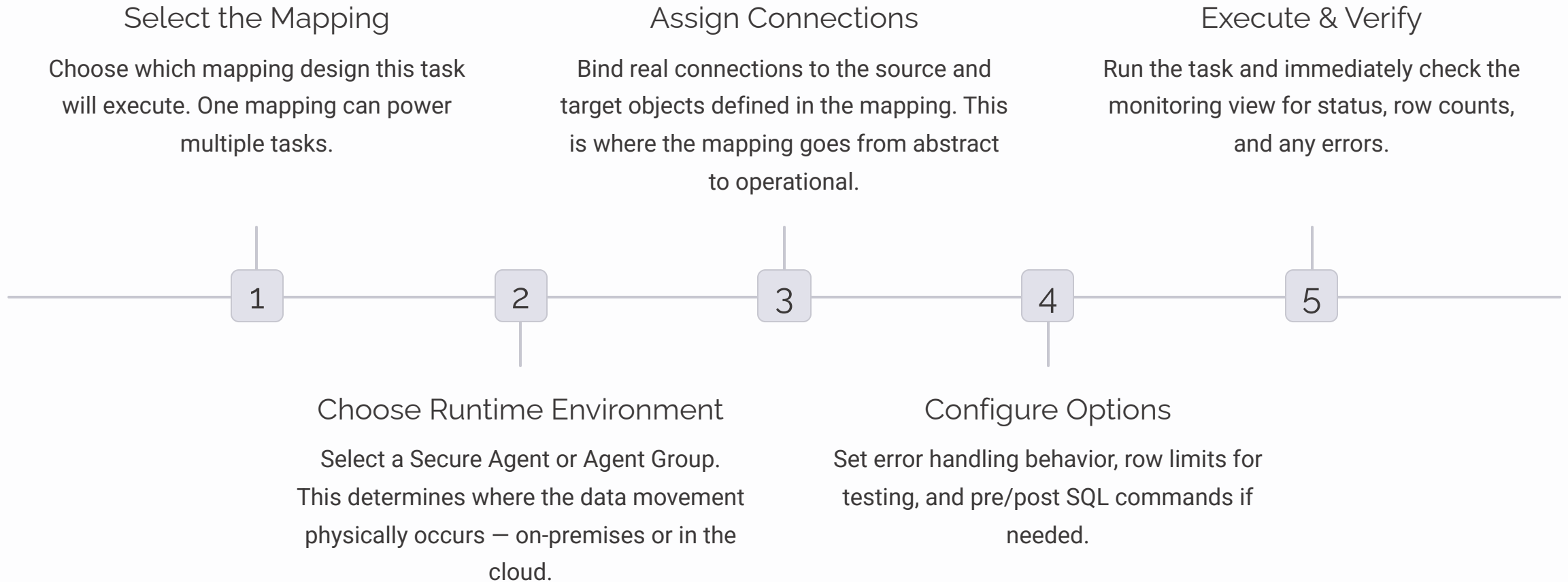


Session 9: Mapping Tasks

Your mapping is designed and validated. Now it's time to **run it**. A Mapping Task is what brings a mapping to life — it ties your blueprint to a specific runtime environment, source and target connections, and execution configuration. This is what you schedule, monitor, and hand off to operations teams.

Creating & Configuring a Mapping Task

Creating a Mapping Task is straightforward, but each configuration choice matters. Here's what you define when setting one up:



Session 10: Basic Parameterization

Hard-coding values in a mapping is a trap. Imagine building a mapping that filters orders from "2024-01-01" — next month you have to edit the mapping itself to change the date. **Parameters** solve this elegantly: define the value once as a named parameter, then change it at runtime without touching the mapping logic. This is the foundation of reusable, maintainable pipeline design.

What to Parameterize

- File paths and file names
- Filter values (dates, status codes, regions)
- Source and target connection names
- Object/table names for multi-tenant loads

How It Works

Define a parameter like `$$START_DATE` in your mapping. Reference it in your filter condition: `order_date >= $$START_DATE`. When creating the Mapping Task, supply the actual value. Each task run can use a different value — same mapping, different behavior.

Session 11: Monitoring & Troubleshooting

Running a task is only half the story — knowing whether it *succeeded*, *how fast* it ran, and *exactly why* it failed when it does, is the other half. IICS's monitoring tools give you full visibility into every execution. This is the skillset that separates junior developers from engineers operations teams actually trust.



My Jobs View

The central monitoring dashboard. See all running and completed jobs, their status (Success / Warning / Failed), start/end times, and source-to-target row counts at a glance.



Session Logs

Drill into the detailed execution log for any job. Logs show transformation-level row counts, timestamps, and the exact error message and line number for failures. Your first stop when something goes wrong.



Rejected Rows

Rows that fail target constraints (type mismatch, constraint violation, null in required field) are written to a reject file. Review it to understand data quality issues — and decide whether to fix source data or adjust transformation logic.

Day 2 Key Takeaways

You've covered a full data integration workflow today – from blank canvas to monitored execution. Here's the mental model to take with you:



Monitor & Troubleshoot

Use logs and reject files to own your pipelines confidently



Execute with Mapping Tasks

Bind mappings to runtime environments and run them



Transform Data

Expression, Filter, Lookup, Aggregator – your core toolkit



Design Reusable Mappings

Parameterize, validate, preview before you execute

📌 The best ETL developers think in reusable patterns. Every mapping you build today is a template for the next ten you'll build tomorrow.